

Naruto 5e - System Architecture Specification

(Initial Draft)

Companion to `naruto5e_rules_spec.md` (the rules/tool-surface design) and `naruto5e_build_audit.md` (build readiness). ****This document supersedes the earlier framing where the engine and the MCP tool surface were one thing.**** They are now three separately-hosted tiers. Local-first, non-commercial, tunnel-shared (see the spec's Attribution & Deployment section).

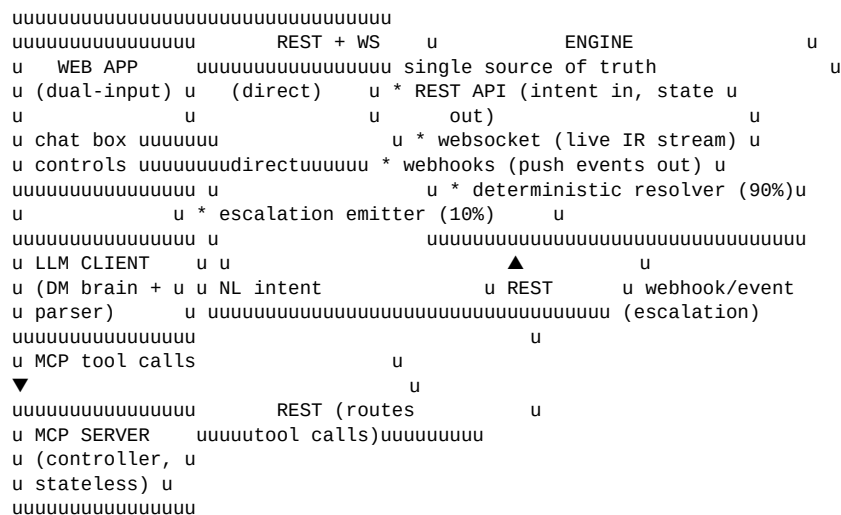
0. THE DECISION (what changed and why)

Originally the system was conceived as a single "naruto5e-mcp" - the MCP tools were the engine. That blurs a surface with a system and bets everything on one protocol. We now separate concerns into three independent, separately-hosted tiers:

1. The Engine - the authoritative game system. Protocol-agnostic. Exposes a RESTful API, webhooks, and a websocket. Knows nothing about LLMs or MCP. It is the asset.
2. The MCP Controller - a thin, stateless adapter that exposes engine capabilities as MCP tools and routes tool calls into the engine's REST API. Holds no game state.
3. The LLM Client - connects to the MCP server (which connects to the engine). It is the primary intent parser and the narration/adjudication brain.

Guiding principle (carried from day one): ***the engine owns truth; clients submit intent; renderers are views.*** MCP is now one consumer* of the engine, not the engine itself. If MCP changes or is replaced, the engine is untouched.

1. TIER OVERVIEW



Two kinds of player input, two routes, one engine:

- * Natural-language intent (the main experience): player -> LLM client -> parsed -> MCP -> engine.
- * Direct manipulation (deterministic shortcuts): UI control -> engine REST directly.

The engine cannot tell (and does not care) whether a structured intent came from an LLM parse or a button click. Intent is intent. It validates and resolves identically. This is the symmetry principle expressed in real protocols.

2. THE ENGINE (tier 1)

The authoritative game server. Stateful, protocol-agnostic, the single source of truth for characters, combat, jutsu, standing, narrative, and scene/spatial state. Implements the corrected 20-entry playable clan roster, at least 8 confirmed base classes pending ninth-class reconciliation, and all the tool families from naruto5e rules spec.md as internal services, exposed over HTTP.

2.1 Responsibilities

- * Hold and persist all game state (Postgres-backed).
- * Accept structured intent from any authenticated client via REST.
- * Run the deterministic resolver (~90%): dice, TurnBudget affordability, chakra/component gates,

movement/LoS, condition ticks, standing checks, clash resolution, and elemental-advantage checks - all the code-resolvable rules.

- * Escalate the contested/novel ~10% out to the LLM tier for a ruling, then apply the ruling as a new intent.
- * Emit the IR stream (typed events) over websocket to all subscribed clients (renderers).
- * Fire webhooks for out-of-band events (escalation-needed, mission-complete, etc.).
- * Enforce roles/permissions server-side (player vs DM; a player client cannot emit a DM intent).

2.2 Surfaces

REST API - the single intent-submission + state-query surface. Representative endpoints:

POST	/v1/rooms/{roomId}/intent	submit a structured intent (the universal seam)
GET	/v1/rooms/{roomId}/state	full room state snapshot (for client hydration)
GET	/v1/characters/{charId}	character record
POST	/v1/characters	create/build (character_manage)
GET	/v1/rooms/{roomId}/encounter	combat state
POST	/v1/rooms/{roomId}/encounter/advance	advance turn
GET	/v1/jutsu	jutsu catalog (read model)
GET	/v1/adversaries / POST .../spawn	Bingo Book / adversary_manage
... (one resource per tool family in the spec; every mutation is an intent)		

Websocket - /v1/rooms/{roomId}/stream. Pushes the live IR event stream to renderers: {type:"move"|"attack"|"cast"|"damage"|"condition"|"down"|"advance"|"narrate"|...}. This is what

the web app's visualizer and narration feed subscribe to.

Webhooks - engine -> outbound HTTP to registered listeners (the LLM tier). Used for events that need an external decision or async reaction:

escalation.needed, intent.contested, mission.completed, standing.changed, turn.npc (an

NPC/adversary's turn needs the LLM to declare its action).

2.3 The resolution loop (where the LLM touches both ends)

1. Intent arrives (POST /intent) - from MCP (LLM-parsed) or web app (UI-generated).
2. Engine validates: role allowed? budget affordable? chakra/components? legal target?
-> if invalid: reject with reason (deterministic).
3. Engine attempts deterministic resolution:
-> ~90%: resolve in code, mutate state, emit IR over websocket. DONE.
-> ~10% (contested/novel/ambiguous): fire `escalation.needed` webhook with the situation.
4. LLM tier receives escalation, rules, and POSTs the ruling back as a new intent.
5. Engine applies the ruling, emits IR. DONE.

The LLM is at the front (parsing NL -> intent) and the back (adjudicating escalations); the engine's deterministic core sits in the middle. Clean, inspectable loop.

Corrected source-grounded resolver modules to expose internally:

- * clash_resolve - detects/receives clashing jutsu interactions and resolves the opposed contest.
- * elemental_advantage_resolve - applies Fire > Wind > Lightning > Earth > Water > Fire superiority.
- * roster_validate - validates the corrected 20 playable Chapter 2 entries and separates NPC/adversary bloodline/release tags from player clan choices.
- * class_roster_validate - supports the 8 confirmed Chapter 4 base classes and keeps a hard TODO for the rulebook's unresolved "nine classes" statement.

2.4 What the engine is NOT

- * Not an LLM host. No model calls live here.
- * Not MCP-aware. It has never heard of MCP.
- * Not a renderer. It emits IR; drawing happens in clients.

3. THE MCP CONTROLLER (tier 2)

A thin, stateless adapter. Its entire job: expose the engine's capabilities as MCP tools and route each tool call to the corresponding engine REST endpoint. The tool surface designed across naruto5e_rules_spec.md is the contract this controller implements - not engine internals.

3.1 Responsibilities

- * Advertise MCP tools (character_manage, jutsu_manage, combat_action, mission_manage, adversary_manage, standing_manage, narrative_manage, ...) to the connected LLM client.

- * On a tool call: translate args -> engine REST request -> return the engine's response verbatim (or lightly shaped to the MCP result format).
- * Carry auth context (which room, which actor, which role) so the engine can enforce permissions.
- * Hold no game state. Restartable at any time with zero data loss.

3.2 Why thin matters (it is now the hot path)

Because the LLM is the primary intent parser, most play flows through this controller. So:

- * Keep per-call work minimal (no parsing, no game logic - the LLM already parsed; the engine resolves). The controller is a forwarder.
- * Latency and reliability matter here; it's not a side door.
- * It is still trivially replaceable - if MCP is swapped for another protocol, only this tier changes.

3.3 Mapping (tool -> endpoint), illustrative

MCP tool call	-> engine REST
character_manage.create(...)	-> POST /v1/characters
combat_action.attack(...)	-> POST /v1/rooms/{room}/intent {type:"attack",...}
jutsu_manage.cast(...)	-> POST /v1/rooms/{room}/intent {type:"cast",...}
standing_manage.spend_favor(...)	-> POST /v1/rooms/{room}/intent {type:"spend_favor",...}
adversary_manage.spawn(...)	-> POST /v1/rooms/{room}/adversaries/spawn

Note: nearly every mutation tool collapses to "submit an intent." Query tools map to GETs.

4. THE LLM CLIENT (tier 3)

The DM brain. Connects to the MCP server; through it, to the engine. Drives most of the experience.

4.1 Two jobs

1. Intent parser (front of loop). Turns player natural language ("I leap off the bridge and hurl ice senbon at the masked nin") into one or more structured tool calls. This is the primary interaction path for players.
2. Adjudicator (back of loop). Subscribes to the engine's escalation.needed / turn.npc webhooks (via the controller or directly). When the engine escalates a contested/novel situation, the LLM rules narratively and submits the ruling back as intent. Also voices NPC/adversary turns.

4.2 Boundaries

- * The LLM never holds authoritative state - it reads room state from the engine when it needs context, and writes only by submitting intent.
- * The LLM does not do arithmetic the engine should do (dice, budgets). It declares *what is attempted; the engine decides what happens*.
- * One LLM-client brain per table/room (the single-adjudicator-per-room rule from the architecture work). Scaling = more rooms, each with its own brain pointed at its engine room.

5. THE WEB APP (dual-input client)

The role-aware dashboard (sheet, party, visualizer, narration, jutsu/inventory/notes - per the spec) is a dual-input client of the engine:

- * Chat box -> LLM path. Free-text "what I want to do" goes to the LLM client, which parses it to intent and routes via MCP -> engine. This is the rich, primary mode.
- * Direct controls -> engine REST. Clicking a reachable tile, tapping a known jutsu card, opening a sheet, advancing a turn (DM) - these are already-structured intents; the UI submits them straight to the engine REST API. No LLM tax on deterministic actions.
- * Live updates -> websocket. Every panel subscribes to the room's IR stream and re-renders on events. The visualizer tweens tokens, the narration feed appends prose+rolls, bars update - all from the same IR.

So the visualizer's "click = intent" is the direct path; the narration chat is the LLM path. Same app, two input modalities, one authoritative engine, one shared IR stream.

6. THE INTENT PAYLOAD CONTRACT (the seam everything hangs off)

Both the MCP controller (LLM-parsed) and the web app (UI-generated) submit the same structured intent to POST /v1/rooms/{roomId}/intent. The engine validates and resolves identically.

```
// Intent - the universal submission envelope
{
  "intentId": "uuid",           // client-generated, for idempotency + IR correlation
  "roomId": "uuid",
  "actorId": "uuid",           // the character/adversary acting
  "submittedBy": { "clientId": "llm|ui", "userId": "uuid", "role": "player|dm" },
  "type": "move" | "attack" | "cast" | "use_item" | "ready" | "advance"
  | "spend_favor" | "rest" | "mission_action" | "spawn_adversary"
  | "narrate" | "ruling" | ... , // one per engine-resolvable action
  "params": { /* type-specific, e.g. for "cast": */
    "jutsuId": "uuid", "targets": ["uuid"], "atRank": "C",
    "areaOrigin": { "x": 4, "y": 7 } },
  "cost": { /* optional client hint; engine recomputes authoritatively */
    "action": 1, "bonus": 0, "reaction": 0, "movement": 0, "chakra": 8 },
  "clientTime": "ISO-8601"
}
```

Engine response (synchronous, for deterministic resolution):

```
{
  "intentId": "uuid",
  "status": "resolved" | "rejected" | "escalated",
  "reason": "no action left" | "not enough chakra" | ... , // when rejected
  "events": [ /* the IR deltas produced, also pushed over the websocket */
    { "type": "cast", "actor": "uuid", "jutsu": "Ten Thousand Ice Petals",
      "area": { "shape": "sphere", "radius": 3, "origin": { "x": 4, "y": 7 } }, "cost": { "chakra": 8 } },
    { "type": "damage", "target": "uuid", "amount": 11 } ],
  "escalation": { /* present when status=escalated */
    "situation": "...", "options": [...], "webhookFired": "escalation.needed" }
}
```

Design rules for the contract:

- * The cost field is a hint, never authority. The engine recomputes affordability from the jutsu's casting time + the actor's TurnBudget + chakra. A client cannot spend what it doesn't have by lying in cost.
- * intentId gives idempotency - re-submitting the same intent (network retry) is safe.
- * submittedBy.role is advisory; the engine re-checks permissions against the authenticated session. A UI claiming role:"dm" doesn't make it so.
- * Every resolved intent yields events[], which are exactly the IR pushed over the websocket - so the submitting client and all observers converge on identical state.

7. HOSTING & SCALING (local table)

Three processes per deployment, plus a database and the front end:

```
[ Postgres ] uu [ ENGINE ] uuwebsocketuuu [ WEB APP ] (served static; players' browsers)
  ▲      u
  REST u    u webhook
  u      ▼
  [ MCP SERVER ] uuMCPuu [ LLM CLIENT ] (the AI DM)
```

- * A table = one engine room (one writer / one adjudicator per room - the consistency boundary).
- * Humans join via the web app (browser -> engine REST + websocket).
- * The AI DM is the LLM client (-> MCP -> engine).
- * Tunnel (Tailscale/Cloudflare) exposes the web app + websocket to trusted, invited players.
- * Scale by running more engine rooms (subdivide/sharding), each with its MCP server + LLM client pointed at it. Never multiple adjudicators in one room. The MCP server and LLM client are

stateless/per-room and cheap to spin up.

- * Drop the heavy public-multi-tenant concerns (billing, abuse, strangers-auth) - local + trusted scope keeps it simple, exactly as the deployment decision intends.

8. WHAT THIS REVISES IN THE EXISTING DOCS

- * System map (build audit \$2): add an "exposure" column - every tool family is an **engine internal service** reached via REST; MCP is a consumer, not the host. The web app is a **second* consumer** (direct), not an MCP client.
- * Visualizer & App Shell sections: their "click = intent / chat = narration" split is now

formally the direct-REST vs LLM-MCP routing; the IR stream is the engine websocket.

- * Adjudication (parse/resolve/escalate): parsing moves to the LLM client (tier 3); resolving stays in the engine (tier 1); escalation is an engine webhook back to tier 3.
- * Attribution/deployment: unchanged in intent; now expressed as three hostable processes behind a tunnel.

Open items to pin next

- * Exact webhook vs. subscription choice for turn.npc and escalation.needed (lean: websocket for app IR, lightweight event subscription/webhook for LLM escalations).
- * Auth model between tiers (engine <-> MCP <-> LLM) for the local/trusted scope.
- * The full type enum for intents (one per engine-resolvable action - derivable from the tool surface).
- * Read-model endpoints vs. event-sourced state (does the engine expose snapshots, deltas, or both?).

9. THE WRITE/READ LAW (confirmed - supersedes §0's escalation framing)

The interpretive cleverness lives entirely at the DM (LLM) layer, inbound only. The engine never escalates, never asks for a ruling - it is a pure deterministic resolver of conformed, validated operations.

9.1 The law

- * The DM (LLM) is the SOLE write-path for play. Flow: player declares intent in fiction -> DM validates + conforms it into a legal engine operation -> engine resolves. No player->engine write path exists for play actions.
 - * Players have read-only direct access to their own surfaces (character sheet). They may read without the DM; they may never write.
 - * The combat engine is the sole authority on outcomes - owns all dice and deterministic resolution. DM decides what is attempted; engine decides what happens.
 - * Validation precedes mutation, always. Every state-mutating op is validated (legal? affordable? permitted?) before it resolves. Deep adjudication occurs only at the DM layer.
 - * There is no engine->DM escalation. (Strikes the §0/§2.3 escalation.needed path entirely.)
- The engine is deterministic; the DM conforms novelty into known operations before submission.

9.2 Access = exposure, not authentication

There is no credential challenge. Access is governed by which tool surfaces are exposed to whom:

- * Player client: read-only surfaces (own sheet, party glance, shared narration, IR stream). No write tools exposed.
- * DM (LLM) client: the full write surface - every Naruto 5e operation. This is the only client that can mutate play state.
- * The engine enforces this by what it hands each client, not by checking passwords. Read-capable vs. write-capable surfaces is the permission model.

9.3 The action surface = the ruleset

The set of submittable actions is exactly the Naruto 5e ruleset specced in naruto5e_rules_spec.md. There is no separate enum to invent: any action available in Naruto 5e is the

surface (move, attack, cast jutsu by classification/rank, use item, rest, downtime activities, standing operations, combat actions, adversary ops, jutsu-builder, ...). The engine's vocabulary = the rulebook's mechanics.

9.4 State reads = scoped queries

A client asking the engine for state asks a specific, scoped question (this character / this encounter / this room). The engine answers exactly that. Not a firehose, not an abstract snapshot-vs-stream choice - each read is a bounded request, with scope built into the query. The live IR websocket then streams deltas against whatever scope the client is observing.

9.5 Design imperative: DM surface ergonomics are first-class

Because the DM is the live write-path, the tool surface MUST be **fast to navigate and natural to narrate from**. Every operation should be a single legible call returning a result the DM can fold into prose immediately. A human-feeling DM cannot fumble a clumsy API mid-scene. Ergonomics of the surface are a hard requirement, not polish - operations are designed for one-call composition and narration-ready returns.

9.6 Resolved open items (from §8)

- * ~~Webhook vs. subscription for escalation~~ -> N/A. No escalation path exists.
- * Auth model -> exposure-based, not credential-based (§9.2).
- * Intent type enum -> = the ruleset (§9.3); no separate enumeration.

* Snapshot vs. event-stream -> scoped queries + IR delta stream (§9.4).

10. THE BATCH TOOL (batch) - sequenced multi-op submission in one pass

The capstone of the DM-ergonomics imperative (§9.5). A single narrated beat is usually many engine operations; the DM should submit them as one ordered list and get all results back in one pass - not call -> wait -> call -> wait -> call -> wait.

10.1 What it is

batch takes an ordered list of any length of other tool calls (intents/operations) and executes them as one sequenced transaction against evolving state, returning all outputs together.

```
batch(ops:[
  {type:"move",    actor:"yuki", to:{x:6,y:3}},
  {type:"cast",    actor:"yuki", jutsu:"Ten Thousand Ice Petals", area:{sphere:20}, origin:{x:7,y:4}},
  {type:"advance"}
])
-> returns ordered results: [ move-IR, cast-IR + 2xsave + 2xdamage + 1xdown, advance-IR ]
```

One narrated sentence ("Yuki dashes in, looses the ice storm dropping a bandit, and the turn passes") = one batch call = one fluid narration beat.

10.2 Sequenced, NOT parallel (the key property)

Operations are dependent and ordered: a cast's chakra deduction happens before the next op's affordability check; a save result gates damage; damage gates the down-check. The engine runs the list in order, each step seeing the state the prior step left. It is an ordered transaction, not a parallel fan-out.

10.3 Failure semantics (the one real decision - DECIDED)

Default stop-on-failure: execute in order until a step is invalid; commit everything up to that point; return the failure reason + the remaining unexecuted steps for the DM to re-conform. Optional atomic:true flag -> all-or-nothing: any invalid step rolls back the whole batch (nothing

mutates) - for beats where a partial application would be incoherent (don't spend the chakra if the jutsu's targets turn out illegal). (Best-effort/skip-invalid is intentionally NOT offered - it leaves half-applied beats the DM didn't intend.)

Rationale: under the write/read law the DM has already validated+conformed the beat before submitting, so mid-batch failures are rare; stop-and-report is the right default, atomic is the safety valve.

10.4 Honors every existing rule

- * Affordability gate (TurnBudget + chakra + components) runs per-step against evolving state.
- * Write/read law: a batch is the DM submitting - a legal write-path; players never batch writes.
- * Validation precedes mutation per step, in order.
- * IR ordering: a batch emits an ordered IR stream in execution order, so renderers play the beat correctly (visualizer: tween move -> draw AoE -> drop bars -> fell token; narration feed appends rolls in sequence). The batch's outputs ARE that ordered IR, returned to the DM and streamed to observers identically.

10.5 Why it's an economic/ergonomic requirement, not just convenience

Without batching, a 6-op beat is 6 sequential round-trips - directly violating §9.5 (the DM can't fumble a clumsy API mid-scene). With it, the DM resolves and narrates a whole turn in one pass. It is the mechanism that makes "navigate the surface quickly, integrate results naturally into narrative" actually achievable. It belongs to the DM write-surface and shares the deterministic engine - every batched op is still a normal, validated, IR-emitting operation; batch only changes how many are submitted per pass.

11. EDUCATIONAL FAILURES (the engine's rejection contract)

Stop-on-failure is confirmed as the batch default (§10.3). Beyond that: **every failure the engine returns is educational** - a redirect, not a dead end. This applies to all* rejected operations (single or batched), and it exists because the DM is an LLM conforming fiction into operations live; a terse error forces blind retry, an educational error lets it re-conform correctly and narrate the correction naturally.

11.1 The four parts of every failure return

1. What failed - the exact operation + step index in the batch (e.g. "op 3 of 5: cast").
2. Why - the specific named rule with the actual numbers, never a generic code. Not "invalid action" but: "Ten Thousand Ice Petals costs 8 chakra; Yuki has 5 remaining after the move." Cite the rule and the values.
3. State at stop - what committed before the failure (ops 1-2 applied; chakra now 5; position now (6,3)), so the DM knows exactly where the world stands.
4. How to fix - the remaining unexecuted ops + a concrete actionable suggestion where one exists ("cast at a lower rank for 5 chakra, or take a different action"). This is what converts a rejection into a redirect.

11.2 Shape

```
{
  "status": "rejected",
  "failedAt": { "index": 2, "op": {"type": "cast", "jutsu": "Ten Thousand Ice Petals"} },
  "reason": {
    "rule": "chakra_affordability",
    "explain": "Ten Thousand Ice Petals costs 8 chakra; actor has 5 remaining after prior ops.",
    "values": { "required": 8, "available": 5, "shortfall": 3 }
  },
  "committed": [ /* IR of ops 0-1 that DID apply */ ],
  "stateAfter": { "chakra": 5, "position": {"x": 6, "y": 3}, "budget": {"action": 0, "bonus": 1} },
  "remaining": [ /* ops 2-4, unexecuted, for re-conforming */ ],
  "suggestions": [
    "Cast at a lower rank (<=5 chakra)",
    "Substitute a bonus-action jutsu (1 bonus available)",
    "Take a different action"
  ]
}
```

11.3 It surfaces what the affordability gate already knows

The affordability gate (TurnBudget + chakra + components) already computes the precise reason a thing can't happen. Educational failure is largely ****surfacing that reason in structured, narration-ready form**** rather than collapsing it to a generic rejection. Low marginal cost, high ergonomic payoff.

11.4 The fiction loop

An educational failure feeds straight back into narration: the DM reads "no chakra left after the dash" and narrates "Yuki reaches for the ice storm but her reserves are spent from the sprint -

instead she...", then submits the re-conformed beat. The failure becomes a story beat, not a stall. This is the §9.5 ergonomics imperative applied to the unhappy path: the surface must be as fast and natural to recover from as it is to act on.

11.5 Principle

No bare rejections anywhere in the engine. Every "no" carries what/why/where/how-to-fix. A rejection that doesn't teach the DM how to proceed is a spec violation.

12. THE TICK SYSTEM - rest-bounded multi-agent world advancement

Time advances only at the moments the fiction already pauses: short rest, long rest, downtime. A tick is not a continuous clock - it is a multi-agent off-screen turn fired at a rest boundary, where the NPCs who populate the world act, mediated by the DM, surfaced to players only when it touches them.

12.1 The three laws (confirmed)

1. Every important NPC / NPC-group is an LLM agent. Major NPCs (Kage, jonin-sensei, rival, clan elder) and groups (a patrol, a rogue cell, a faction) each have an agent representing them. On a tick, the engine calls each relevant agent to decide what it does during the elapsed time. The world during a tick is populated by agents, not just rules.
2. The DM sees all and is the sole surface into the engine. NPC agents never write directly. Each returns declared intent -> the DM conforms + prices it into engine operations (the write/read law, §9) -> engine resolves deterministically. NPC agents propose; the DM disposes into the engine.
3. Off-screen ticks do not surface to the player unless they affect the player. The world fully advances (NPCs act, Standing shifts, plots move); the engine resolves all of it; but ****narration is filtered to player-relevant outcomes.**** A rival's silent background training is invisible until it

becomes a player-facing consequence; a robbed safehouse surfaces because it is the player's.

12.2 Tick scope keyed to rest type (ties to Ch.7)

Trigger	Tick magnitude	What moves
Short rest rotation, a wounded enemy flees); heat barely shifts	small	immediate-vicinity agents; short-horizon (guard
Long rest cools one step; a patrol passes	medium	the day's events; nearby agents travel/act; heat
Downtime (weeks) completes; patrons act; reputations shift; background missions resolve; rivals advance	large	the whole standing/economy/rogue web; training

Bigger time-skip -> more agents called -> more world movement. Magnitude bounds which agents are called and how far their horizon reaches.

12.3 The tick loop

1. Rest/downtime declared (player intent, via DM) -> engine identifies the tick magnitude.
2. Engine SELECTS the agents in scope (by proximity + stake + magnitude) and CALLS each: agent.tick(elapsed, worldContext, ownMemoriesOfPlayers, ownStanding, ownGoals)
-> each returns declared intent (what this NPC/group does during the elapsed time).
3. DM receives ALL agent intents (sees everything), conforms + prices each into engine ops, and submits them - typically as a single `batch` (\$10), sequenced.
4. Engine resolves deterministically: NPC actions priced like any actor's (an NPC can't do in a tick what it couldn't afford in play); routes deltas through the world-consequence systems (npc memory, economy, theft heat-decay, corpse decay) and standing_manage.
5. Player rest benefits apply (HP/Chakra recovery, Will of Fire refresh on mission boundary, etc.).
6. NARRATION FILTER: the DM narrates to the player ONLY the tick outcomes that touch the player. Everything else mutates world state silently (queryable later, but not surfaced).

12.4 NPC agents act through priced intent (fairness)

A tick action is a normal validated, priced operation - an NPC spends its resources/Ryo/Standing just as a PC would. This is what keeps off-screen action fair: the rival who "trains" during your downtime consumes their downtime/Standing to do so; the patron who "calls in a favor" spends standing they hold over you. No agent gets free off-screen power. Pricing = the same engine gates (affordability, the jutsu point model for any improvised NPC action).

12.5 Coupling to the world-consequence systems (\$ in THE RULES)

The tick is the heartbeat that drives the four world-consequence systems off-screen:

- * npc_manage: agents act on their memories of players; new memories + Standing deltas are written.
- * economy_manage: prices drift, gated stock restocks, vendor heat capacity resets.
- * theft_manage: heat decays a step (long rest) or fully (downtime); a witness may report (a patron/jurisdiction agent's tick action) -> Standing penalty; a bounty matures.
- * corpse_manage: a corpse advances a decay stage; an unharvested secret may be taken by another agent during the tick.
- * standing_manage: the net of all agent tick-actions lands as reputation/favor deltas across the per-authority ledgers - the world's power map shifts while the party rests.

12.6 Tool surface

op	params	effect
tick.run declared intents to the DM for conforming	trigger(short/long/downtime), context	selects in-scope agents, calls each, returns their
tick.resolve result set + a player-relevant digest (the narration filter)	conformed batch of NPC + world ops	resolves the tick deterministically; returns full
tick.preview resolving (DM)	trigger	which agents are in scope + their stakes, without

planning)

agent.tick agentId, elapsed, worldContext (per-agent) the NPC agent's declared intent for the elapsed time

tick.run and agent.tick are the only places the engine calls out to LLM agents - and even then,

results return to the DM, who is the sole write-path. The engine itself stays deterministic; the agents live in the LLM tier (tier 3), one brain per important NPC/group, all orchestrated at the rest boundary. The player sees a rested party and a world subtly (or dramatically) changed - but only where it concerns them.

12.7 Why rest-bounded is the right design

Time only moves when the fiction pauses, so there is never "dead air" or an arbitrary background clock. Rest and downtime - already mechanically meaningful (Ch.7) - become the world's pulse. The party's choice to rest is the choice to let the world move. This makes downtime a real strategic decision (the longer you step away, the more the world advances without you), and it makes the world feel autonomous and alive while keeping every off-screen action fair (priced), mediated (DM), and player-respecting (surfaced only when it matters to them).

13. TICK <-> REST/DOWNTIME RETURN PACKAGE (the tick is embedded, not separate)

The tick is not a separate call the DM must fire after a rest. It is ****embedded in the rest/downtime tool's return package.**** One call (rest or downtime) -> one bundled response carrying both the party's rest result and what the world did meanwhile. This honors the ergonomics imperative: the DM rests the party and gets the whole picture back, ready to narrate, in a single return.

13.1 The bundled return (three layers)

A rest/downtime resolution returns:

```
{
  "restResult": {                                // LAYER 1 - direct effect on the resting party
    "type": "short" | "long" | "downtime",
    "recovered": { "hp": +X, "chakra": +Y, "hitDiceSpent": n },
    "willOfFire": "refreshed" | "unchanged",      // refresh on mission boundary
    "downtimeActivity": { "kind":"training", "status":"complete", "result":"learned X" }
  },
  "tick": {                                       // LAYER 2 - the embedded multi-agent world advancement
    "magnitude": "small" | "medium" | "large",
    "agentsCalled": [ ... ],                     // who acted (full record, for world state)
    "resolved": [ ... ],                         // every conformed+priced NPC/world op the engine applied
    "consequenceDeltas": {                       // routed through the world-consequence systems
      "standing": [ {authorityId, repΔ, favorΔ, why} ],
      "heatDecay": [...], "corpseDecay": [...], "economyDrift": [...], "npcMemories": [...]
    }
  },
  "playerDigest": [                              // LAYER 3 - the NARRATION FILTER (tick law §12.1.3)
    // ONLY the tick outcomes that TOUCH the player - what the DM reads aloud.
    "Word reaches you that your safehouse was searched while you trained.",
    "Sensei Rin left a message: the Chunin exams open next month."
  ]
}
```

* Layer 1 is the party's rest benefit (the thing they asked for).

* Layer 2 is the full tick - embedded, resolved as part of this same call (agents called, intents conformed + priced + resolved, consequence systems advanced). Mutates world state.

* Layer 3 is the filtered digest - the only part the DM narrates, per the off-screen surfacing law. Everything else in Layer 2 is silent world-state the engine holds (queryable later via scoped reads, but not surfaced).

13.2 Why embed rather than separate-call

* One narratable beat: "You wake recovered - and word reaches you that..." comes from one return.

* Atomicity: rest benefits and world advancement resolve together; no window where the party has rested but the world hasn't moved (or vice-versa).

* The rest IS the tick trigger: resting/downtime is defined as "let time pass," so the time

passing (the tick) belongs in the same resolution, not a follow-up.

13.3 Tool-surface consequence

rest_manage / mission_manage.downtime (THE RULES, Ch.7) own the tick internally. On

resolution they: apply rest benefits -> fire the embedded tick (§12 loop, scaled to rest magnitude) -> return the three-layer package above. The standalone tick.* ops (§12.6) remain available for the DM to preview (tick.preview) before committing a long downtime, but the normal path is that the tick rides inside the rest/downtime return. The DM never has to remember to advance the world - resting the party does it, and hands back exactly what to say.